

Programmation Backend

PHP et Bases de Données



Pr. ELALAOUI Hasna

h.elalaoui@uca.ac.ma

1

Introduction



Contexte général

- Un site Web dynamique est un site Web dont le contenu change automatiquement. Autrement dit, le contenu visible sur la page n'est pas toujours intégré directement en HTML par le Webmaster.
- Le contenu des sites web dynamiques provient souvent de sources de données. Ces sources peuvent être des fichiers texte, fichiers XML, sources Json, et par dessous tout, il y a les bases de données.
- En effet, les bases de données constituent les sources de données les plus populaires pour différentes raisons. J'en cite:
 - Les bases de données (dites relationnelles) sont structurées sous formes de tables. Chacune des tables contient des données jugées cohérentes, mais complémentées par les données qui figurent dans d'autres tables via des relations.
 - Les bases de données sont faciles à parcourir et interroger, et c'est principalement grâce au langage de requêtes SQL (Structured Query Language).



PHP et bases de données

Afin d'interagir avec une base de données MySQL, PHP propose trois méthodes différentes:

- **Extension MySQL originale:** Il s'agit de l'extension MySQL native au PHP. Pendant longtemps, elle a été pratiquement la seule extension disponible, mais elle est rendue obsolète depuis la version 5.5 de PHP, et a été supprimée en PHP7.
- **Extension MySQLi:** Il s'agit d'une extension un peu plus moderne. C'est d'ailleurs cette extension que l'on va traiter dans ce chapitre.
- **Objet PDO (PHP Data Objects):** L'objet PDO constitue de loin la meilleure solution pour interagir avec une base de données en PHP. Elle dispose de plusieurs avantages, et le plus marquant c'est qu'il permet l'abstraction de la base de données, ce qui a pour effet de pouvoir communiquer avec de nombreux SGBD (et pas que MySQL) tout en maintenant le même code PHP

2

Extension MySQLi



Principe général

- PHP fournit un grand choix de fonctions permettant de manipuler les bases de données. Toutefois, parmi celles-ci quatre fonctions sont essentielles :
 - La fonction de connexion au serveur
 - La fonction de choix de la base de données
 - La fonction de requête
 - La fonction de déconnexion

Afin de créer la base de données sur le serveur, ainsi que les tables et aussi créer les utilisateurs en leur accordant les privilèges nécessaires, il est préférable de se servir de l'outil PHPMysqlAdmin qui propose une interface graphique pour gérer les bases de données MySQL.



Connexion au serveur

- Pour se connecter à une base de données en php, il faut spécifier un nom de serveur, un nom d'utilisateur, un mot de passe et un nom de base. Les fonctions de connexion sont les suivantes:
- **mysqli_connect(\$server,\$user,\$password)** : permet de se connecter au serveur \$server en tant qu'utilisateur \$user avec le mot de passe \$password, retourne l'identifiant de connexion si succès, FALSE sinon
 - **mysqli_select_db(\$base[\$id])** : permet de choisir la base \$base, retourne TRUE en cas de succès, sinon FALSE
 - **mysqli_close(\$id)** : permet de fermer la connexion
 - **mysqli_pconnect()** : idem que mysqli_connect() sauf que la connexion est persistante, il n'y a donc pas besoin de rouvrir la connexion à chaque script qui travaille sur la même base.

les identifiants de connexion ne sont pas nécessaires si on ne se connecte qu'à une seule base à la fois, ils permettent seulement de lever toute ambiguïté en cas de connexions multiples.



Exemple

Connexion

- La première opération à faire si on veut interroger une base de données c'est la connexion au serveur qui héberge celle-ci :

```
$id=mysqli_connect("localhost","root","root");
```

Sélection de la BD

- La fonction `mysqli_connect()` retourne un identifiant de connexion (`$id` dans notre cas) ou la valeur booléenne `false` en cas d'échec.
- Une fois la connexion au serveur établie, on peut désormais sélectionner la base de données à interroger.

```
mysqli_select_db($id,"mabase");
```

- Cependant, on peut résumer les deux opérations que l'on vient de voir dans une seule instruction de la manière suivante:

```
$id=mysqli_connect("localhost","root","root","mabase");
```




Gestion des erreurs

- Souvent, à la connexion, des erreurs peuvent survenir (serveur de base de données à l'arrêt, erreur de login ou de mot de passe...) dans ce cas la fonction **mysql_connect()** retournera false, mais elle générera aussi une erreur PHP de type E_WARNING.
- Un message peut s'afficher ou non sur le navigateur selon la configuration. Or, les instructions qui vont suivre dépendront probablement du succès de la connexion, autrement dit, des erreurs supplémentaires vont survenir.
- Afin d'empêcher cette avalanche d'erreurs, il serait mieux d'arrêter prématurément le script PHP suite à l'erreur de connexion.

Syntaxe

```
$id=mysql_connect("localhost","root","root","mabase") or die("Erreur de connexion");
```

3

Opérations CRUD



Exécution des opérations -1

- Afin d'exécuter une requête SQL via un script PHP, on fait appel à la fonction `mysql_query()`. Par exemple, si on veut insérer un nouvel enregistrement dans une table *personnes*

```
$req=mysql_query($id,"insert into personnes (nom,prenom,login,pass)  
values('Einstein','Albert','a.einstein',md5('2020'))");
```

- La fonction `mysql_query()` accepte deux arguments qui sont, respectivement, l'identifiant de la connexion précédemment établie et la requête SQL à exécuter.
- La fonction `mysql_query()` retourne un identifiant de connexion (`$req` dans notre cas) ou `false` en cas d'échec.
- Si on veut modifier l'enregistrement que l'on vient d'insérer (on va changer le mot de passe par exemple) alors on va exécuter l'instruction suivante:

```
$req=mysql_query($id,"update personnes set pass=md5('ABCD') where id=1");
```



Exécution des opérations -2



Si maintenant, on veut supprimer cet enregistrement alors on exécute le code PHP suivant:

```
$req=mysqli_query($id,"delete from personnes where id=1");
```



L'identifiant `$req` ne nous a été d'aucune utilité pour les trois requêtes exécutées. Donc on peut ne pas le déclarer.

4

Extraction des données



Sélection des données -1

- Bien que la sélection fait aussi partie des opérations CRUD, elle demande quand même un peu plus de traitement.
- Supposons qu'on veut afficher les noms et prénoms de tous les enregistrements qui figurent dans la table personnes (s'il y'en a). Dans ce cas on exécute les instructions suivantes:

```
$req=mysqli_query($id,"select nom,prenom from personnes");
```

- Sous MySQL (comme pour beaucoup de SGBD), le traitement des résultats d'une requête se fait ligne par ligne. Une boucle permettra de recueillir chacune des lignes à partir de l'identifiant de résultat
- Une ligne contient une ou plusieurs valeurs correspondant aux différents attributs retournés par la requête. Ainsi, une ligne de résultat pourra être sous la forme de variables, d'un tableau, d'un tableau associatif, ou d'un objet.



Sélection des données -2



La fonction `mysqli_fetch_assoc()` qui accepte comme argument l'identifiant de la requête (\$req) retourne l'enregistrement courant sous forme d'un tableau PHP (\$tab), ou false s'il n'y a plus aucun enregistrement, ce qui a pour effet d'interrompre la boucle while.



Le tableau PHP \$tab contient autant d'indexs que de champs sélectionnés. Dans ce cas, il contient deux indexs qui correspondent respectivement au nom et au prénom.

```
while($tab=mysqli_fetch_assoc($req)){  
    echo $tab["nom"]." ".$tab["prenom"];  
}
```



Sélection des données -3



mysql_fetch_row(\$result) : retourne une ligne de résultat sous la forme d'un tableau. Les éléments du tableau étant les valeurs des attributs de la ligne. Retourne FALSE s'il n'y a plus aucune ligne.

```
$requete = "SELECT * FROM personnes";  
if($result = mysqli_query($requete)) {  
    while($ligne = mysql_fetch_row($result)) {  
        $nom = $ligne[0];  
        $prenom = $ligne[1];  
        echo "$nom - $prenom<br />";  
    }  
}
```



Ici, on accède aux valeurs de la ligne par leur indice dans le tableau. Une pratique courante est d'utiliser la fonction `list()` pour avoir automatiquement les champs sous forme de variables :

```
while(list($nom, $prenom) = mysql_fetch_row($result))  
{  
    echo "$nom - $prenom<br />";  
}
```




Sélection des données -4



mysql_fetch_array(\$result) : retourne un tableau associatif. Les clés étant les noms des attributs et leurs valeurs associées leurs valeurs respectives. Retourne FALSE s'il n'y a plus aucune ligne

```
$requete = "SELECT * FROM personnes";  
if($result = mysqli_query($requete)) {  
    while($ligne = mysqli_fetch_array($result)) {  
        $nom = $ligne["nom"];  
        $prenom = $ligne["prenom"];  
        echo "$nom - $prenom<br />";  
    }  
}
```



Ici, on accède aux valeurs de la ligne par l'attribut dans le tableau associatif



Sélection des données -5



mysql_fetch_object(\$result) : retourne un objet. Les attributs de l'objet correspondent à ceux de la ligne de résultat. Et les valeurs des attributs de l'objet correspondent à ceux de la ligne de résultat. Retourne FALSE s'il n'y a plus aucune ligne.

```
$requete = "SELECT * FROM personnes";  
if($result = mysqli_query($requete)) {  
    while($ligne = mysqli_fetch_object($result)) {  
        $nom = $ligne->nom;  
        $prenom = $ligne->prenom;  
        echo "$nom - $prenom<br />" ;}  
}
```



Ici, on accède aux valeurs par leur attribut dans l'objet



Autres fonctions utiles

Quelques fonctions supplémentaires très utiles :

- **mysqli_free_result(\$result)** : efface de la mémoire du serveur les lignes de résultat de la requête identifiées par \$result.
- **mysqli_insert_id([\$id])** : retourne l'identifiant d'un attribut clé primaire AUTO_INCREMENT de la dernière insertion.
- **mysqli_num_fields(\$result)** : retourne le nombre d'attributs du résultat.
- **mysqli_num_rows(\$result)** : retourne le nombre de lignes du résultat